

Architettura degli Elaboratori

L'architettura della CPU – Control Unit e nuove istruzioni



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo

Masi,

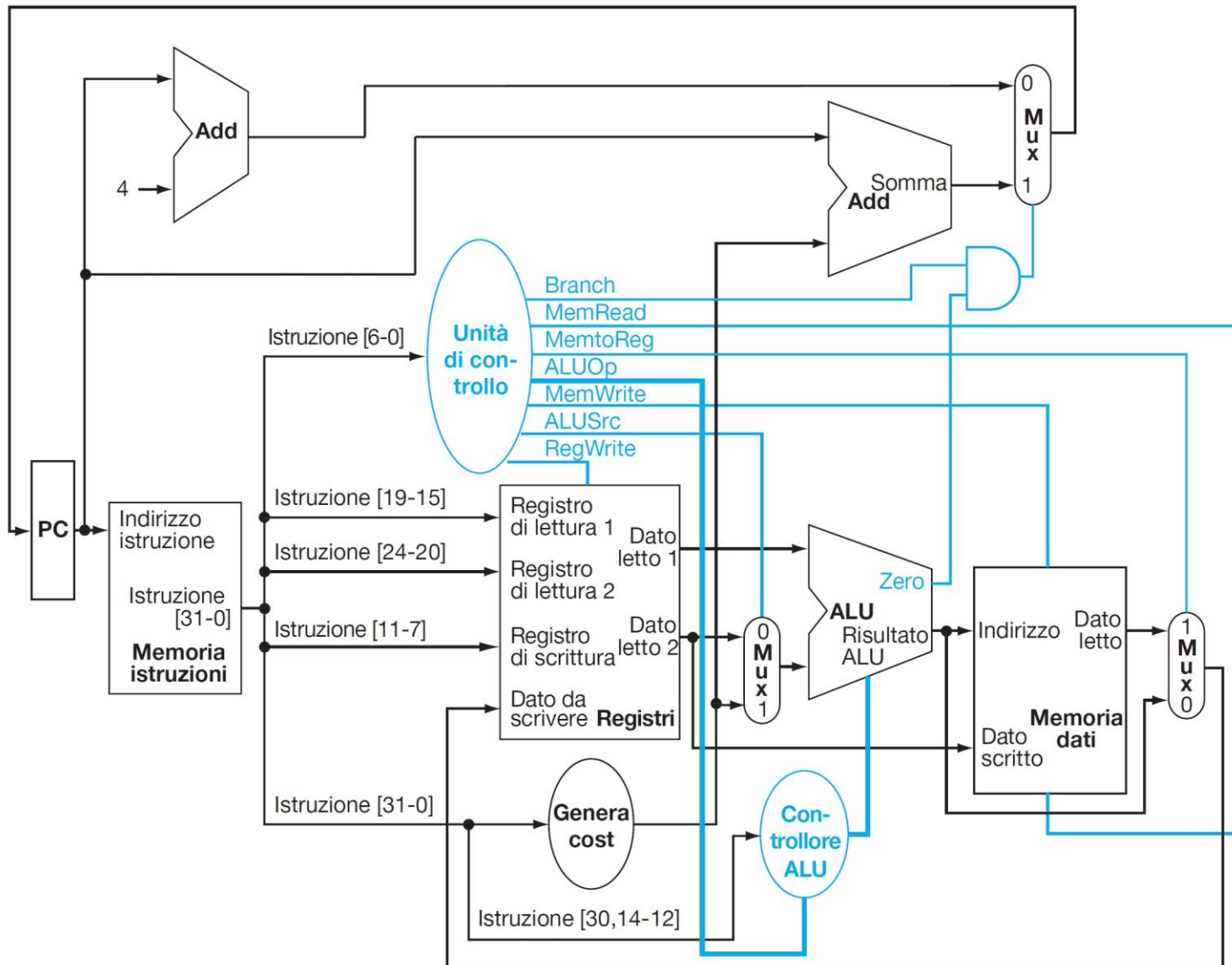
Claudio di Ciccio

[S&PdC]

4.1-4.4

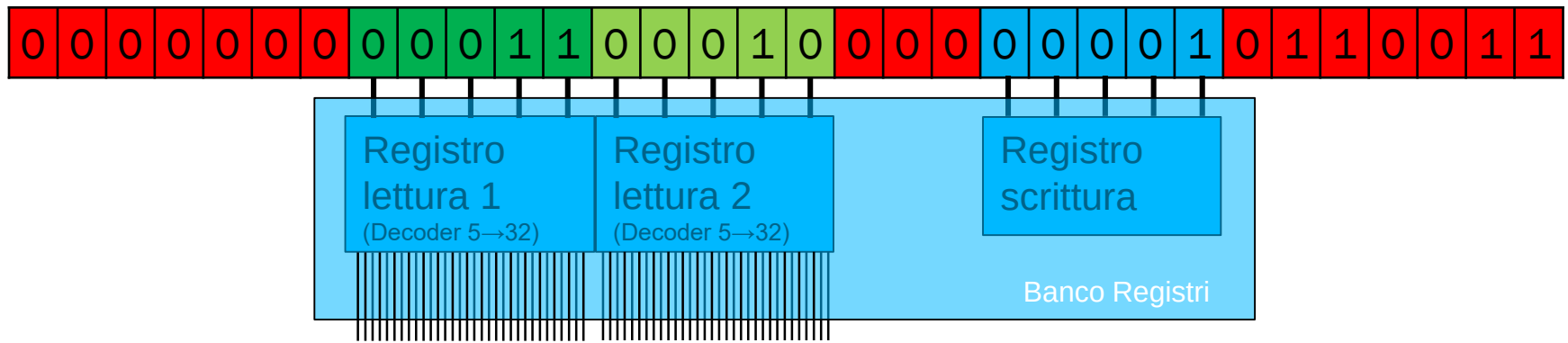
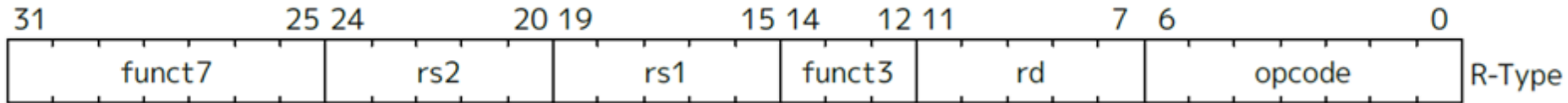


Datapath completo



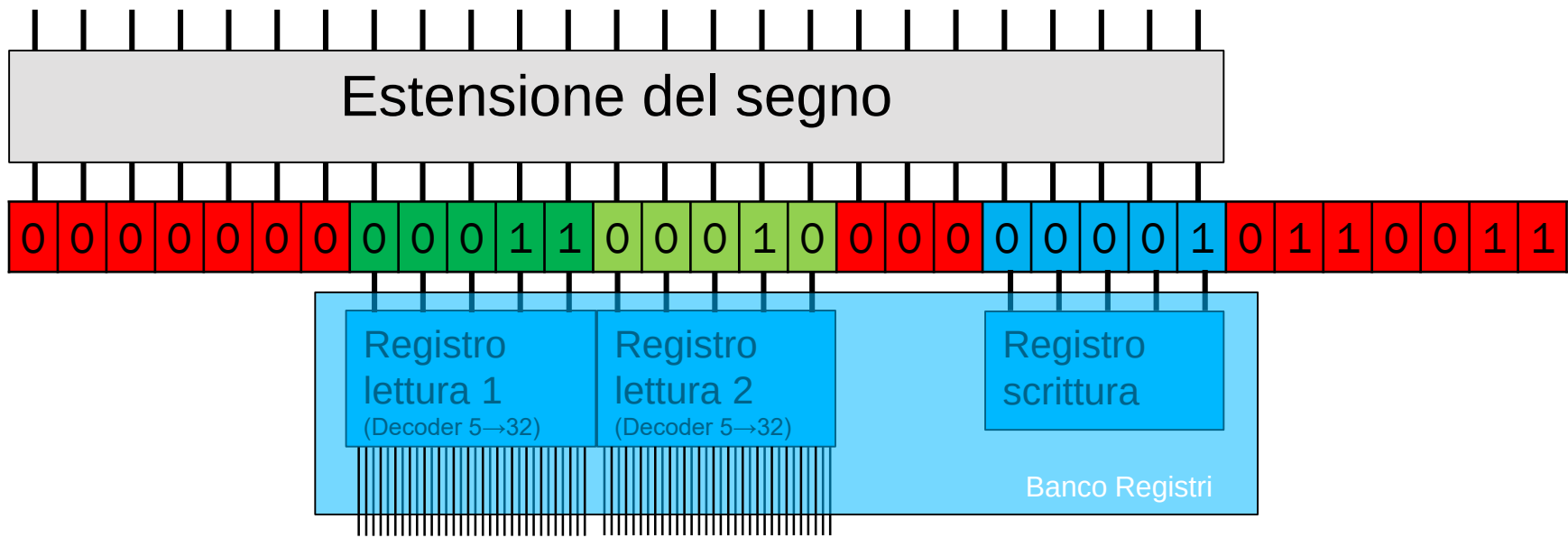
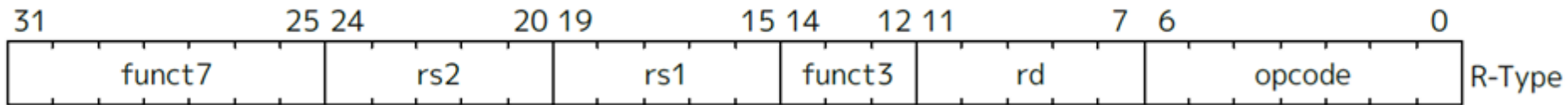
Istruzione – Accesso al banco registri

add x1, x2, x3



Istruzione – Accesso al banco registri + estensione del segno

add x1, x2, x3



Banco registri

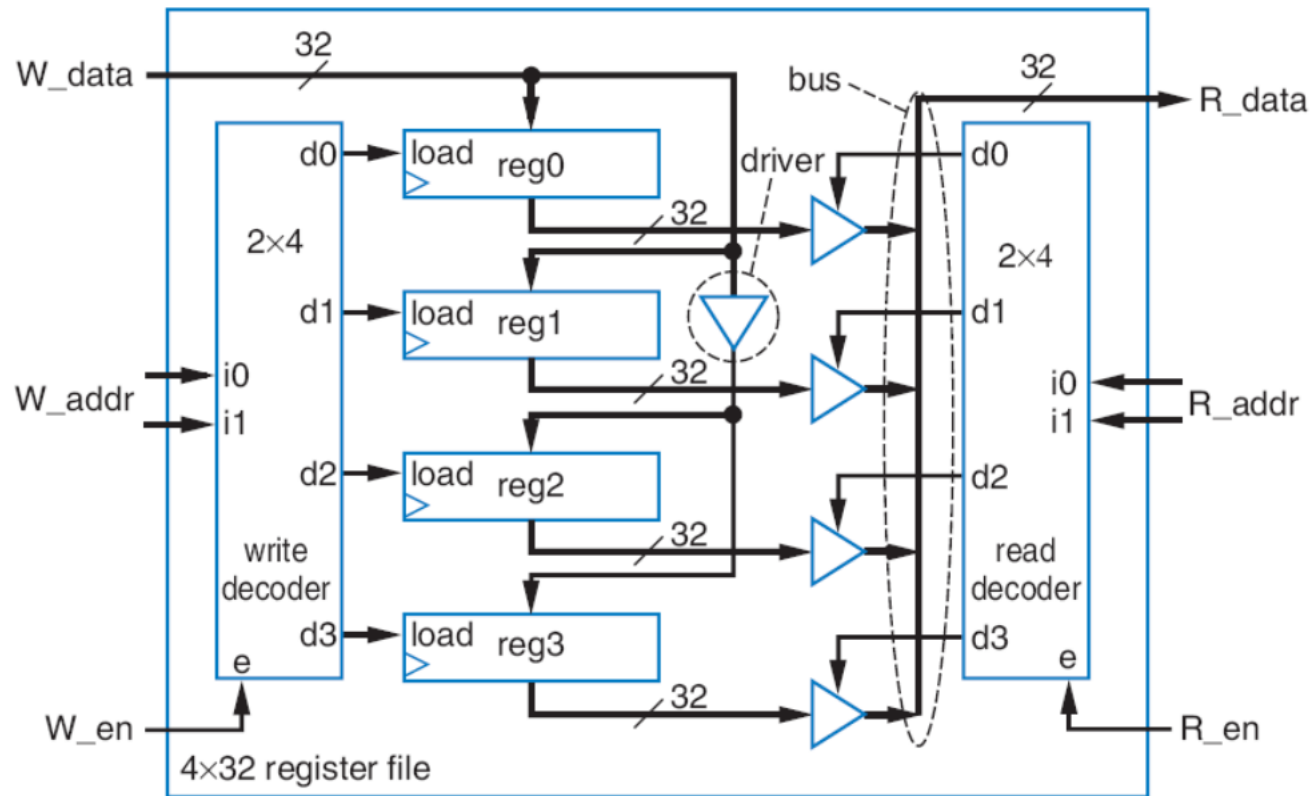
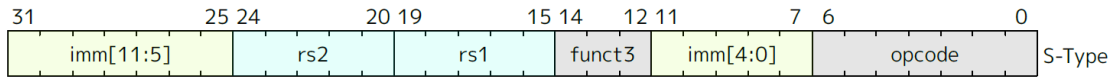
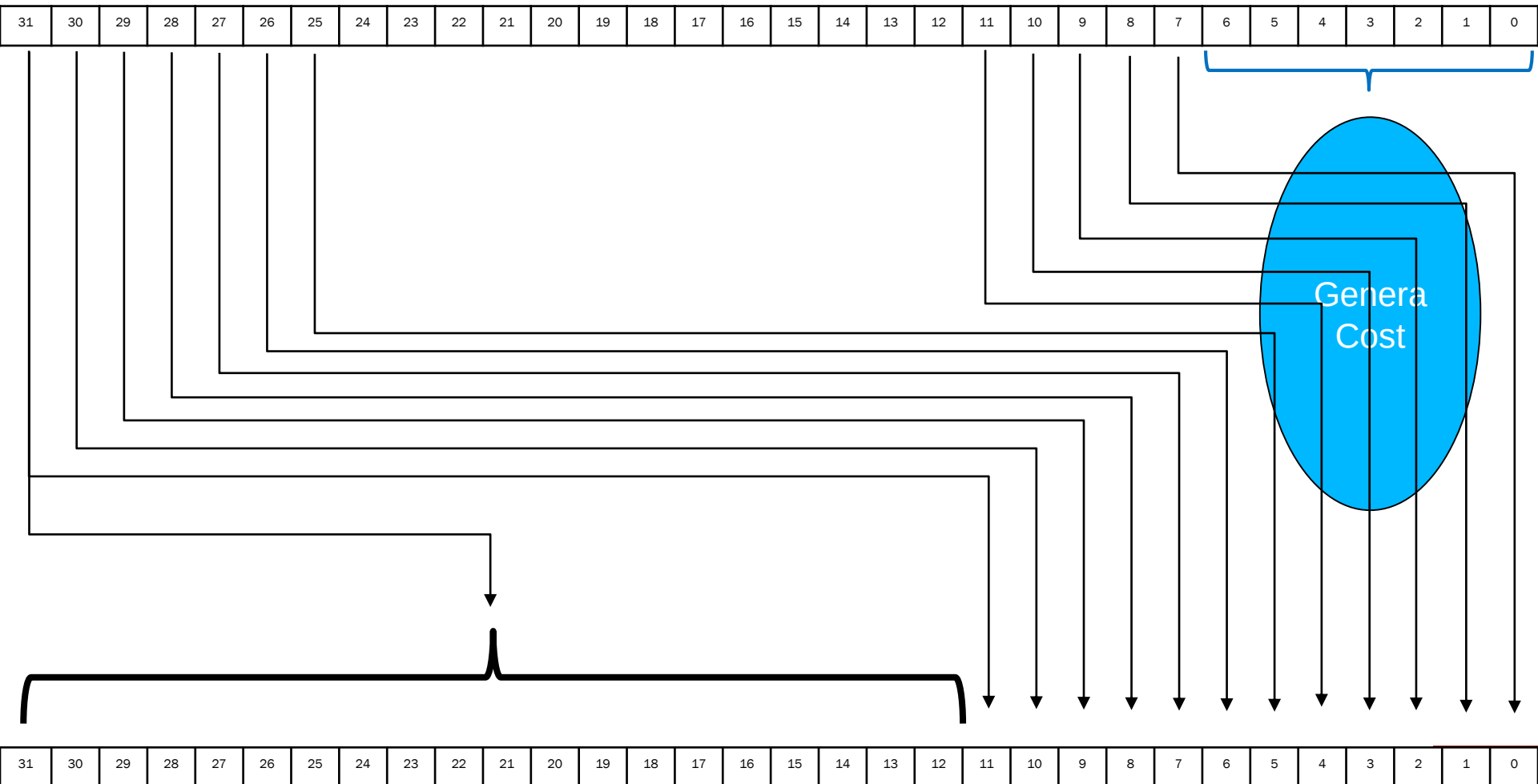


Figure L7-2. Implementation of a $2^2 \times 5$ Register File

Unità di estensione del segno (Genera Cost) esempio – tipo S

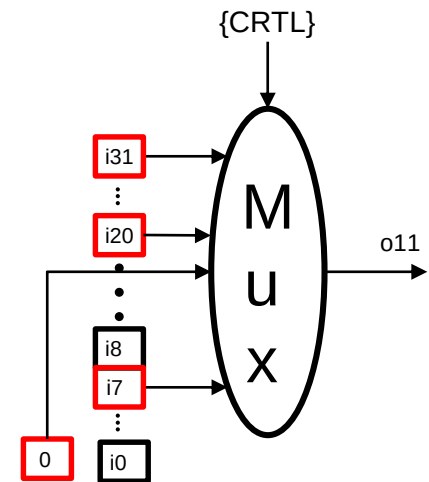
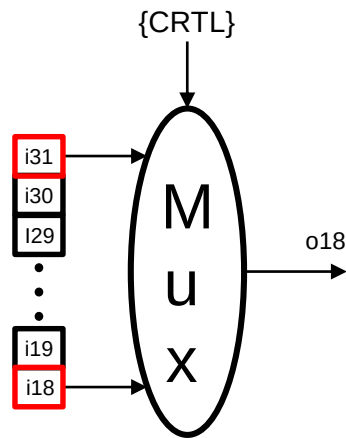
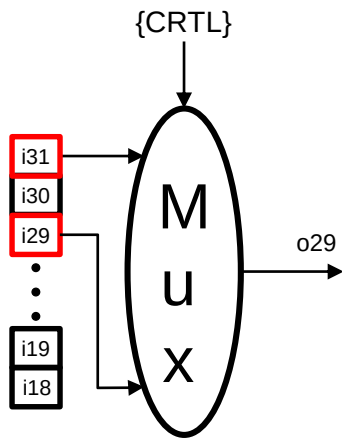


addi rd,rs1,rs2



Estensione del segno, come funziona internamente?

		Campo immediato generato bit per bit																																
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Istruzione	Formato	Input del campo immediato bit per bit																																
Load, Aritm imm	I	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	
Store	S	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i11	i10	i9	i8	i7
Salto cond	SB	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i7	“	“	“	“	“	“	“	“	“	“	0
Salto incond	UJ	“	“	“	“	“	“	“	“	“	“	“	“	“	i19	i18	i17	i16	i15	i14	i13	i12	i20	“	“	“	“	“	“	“	“	“	“	“
Load Upper imm	U	“	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	“	“	“	“	“	“	“	“	0	0	0	0	0	0	0	0	0	0	0	“	
Num input		1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	4	2	2	2	2	2	2	3	3	3	3	3	

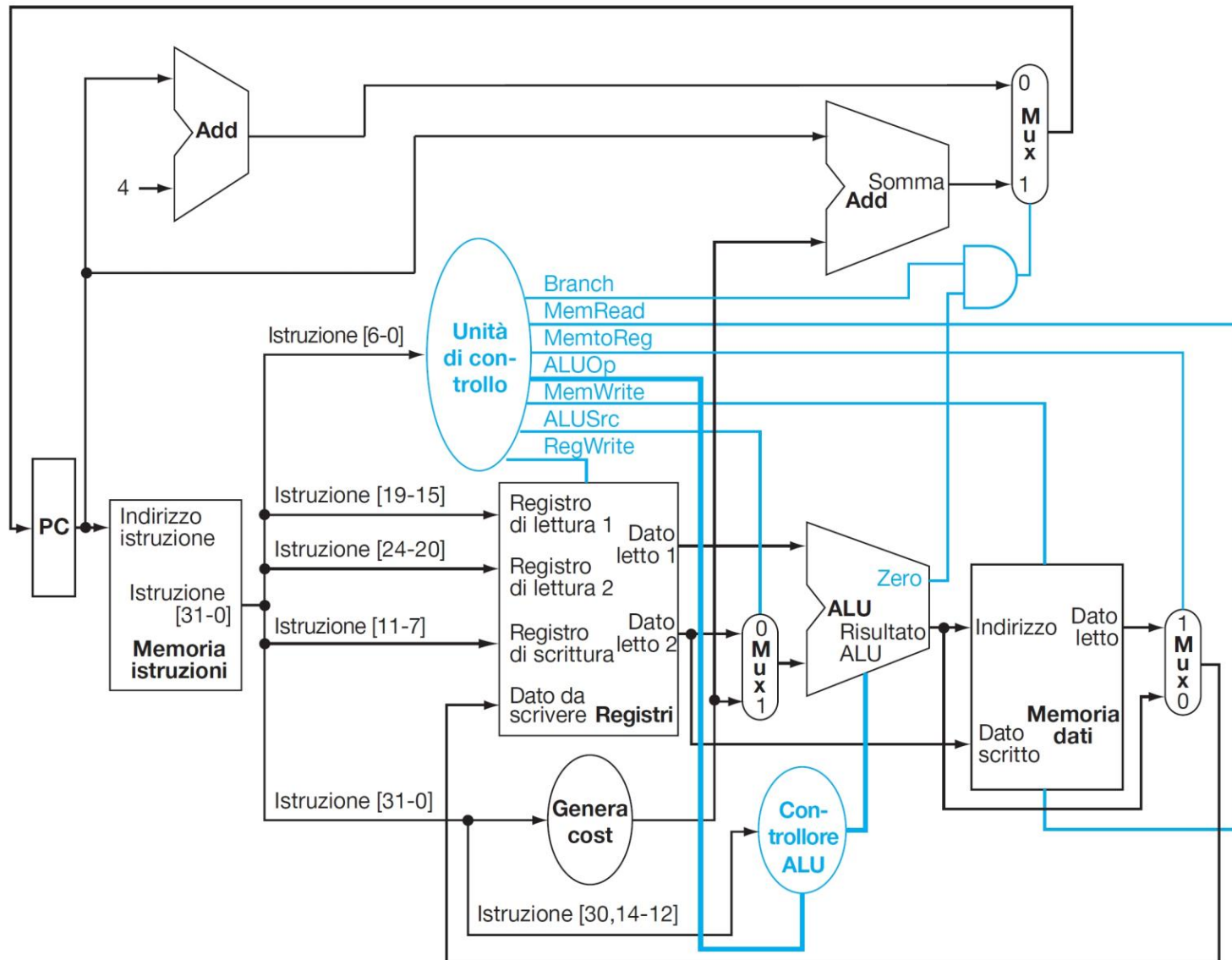


Creazione di nuove istruzioni



SAPIENZA
UNIVERSITÀ DI ROMA

Datapath completo

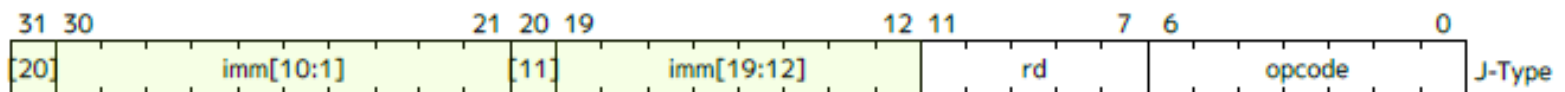
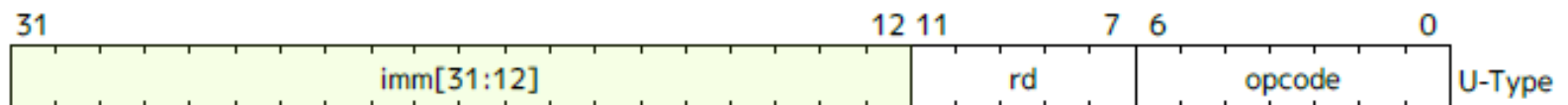
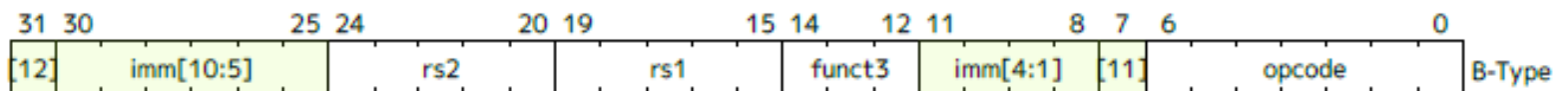
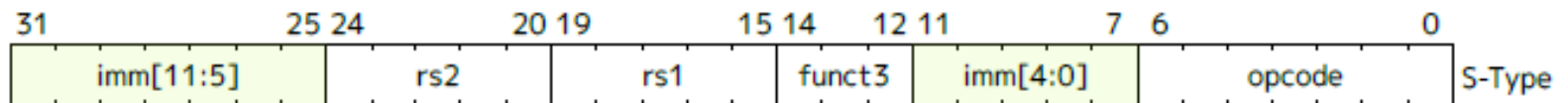
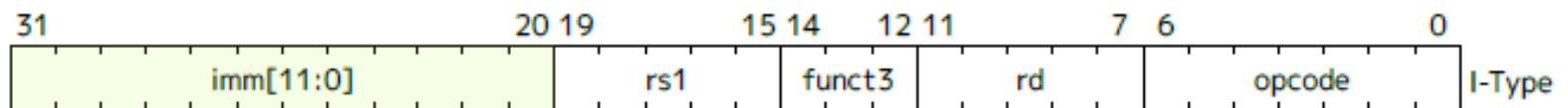
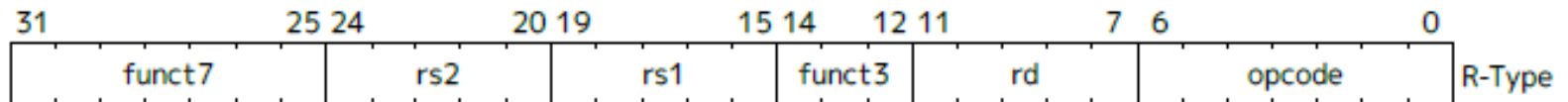


Aggiungere una nuova istruzione

Supponiamo di voler aggiungere la nuova istruzione, J (Jump), dobbiamo:

- Definire la sua **codifica**
- Definire **cosa faccia**
- Individuare le **unità funzionali necessarie** (e se sono già presenti)
- Individuare i **flussi delle informazioni** necessarie
- Individuare i **segnali di controllo** necessari
- Calcolare il **tempo necessario** per la nuova istruzione e se modifica il tempo totale
- è un **indirizzo relativo al PC**
- indica l'**istruzione di destinazione** (deve essere compatibile con tutte le estensioni anche C a 16bit **indirizzamento alla mezza parola**)
- (per i salti tra blocchi diversi sarà necessario introdurre l'istruzione **jr**)

Quale formato scegliere: UJ



- Registro contenente l'indirizzo relativo di salto
- Salto più ampio possibile

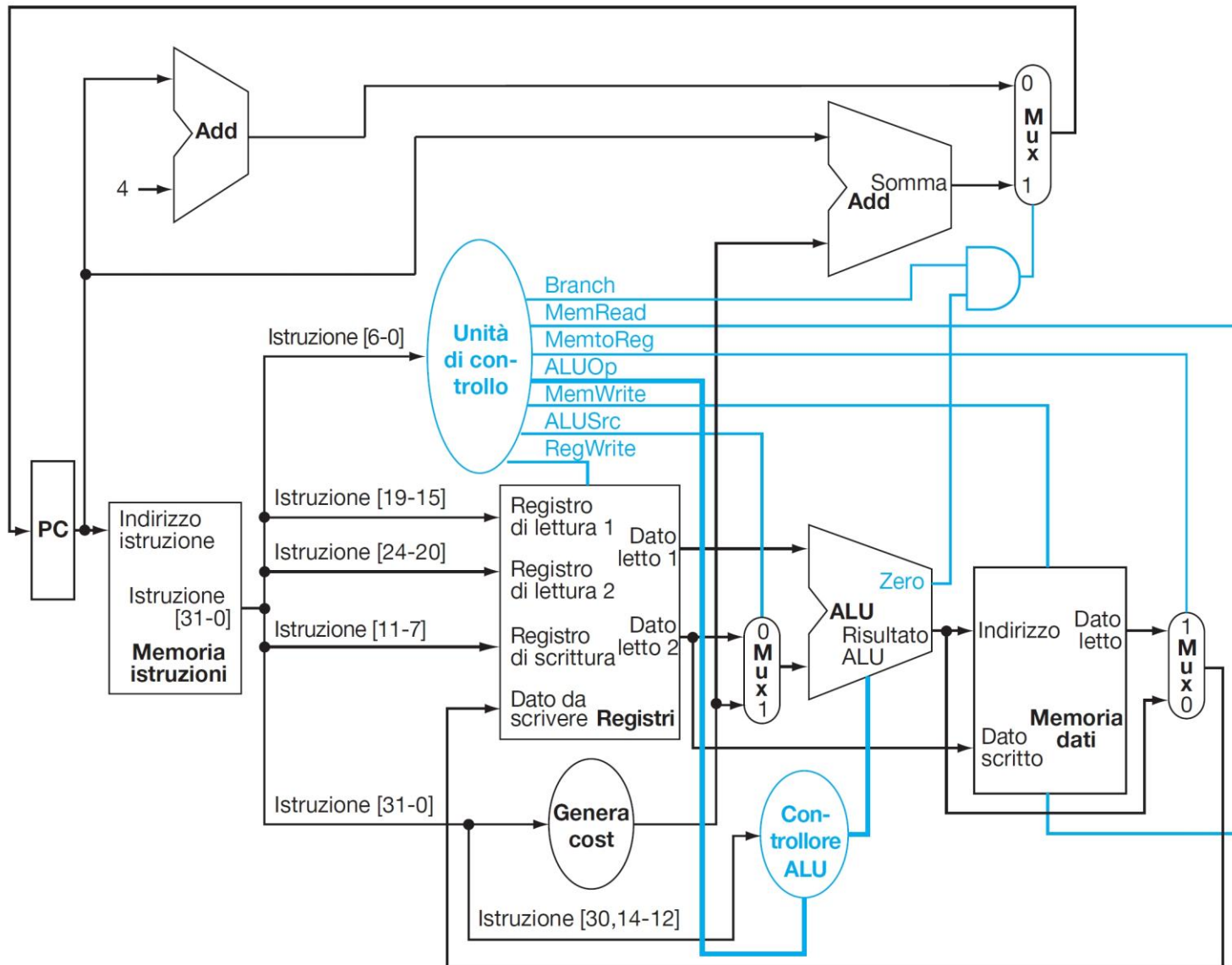
Aggiungere il Jump

Cosa fa:	$PC \leftarrow (\text{shift left di 1 bit di Istruzione}[12-31]) + PC$	
Unità funzionali:	PC	(già presente)
	shift left di 1 bit con riordino dei bit	(con l'unità di est. segno)
	Somma con il PC	(già presente per branch)
	MUX per selezionare il nuovo PC	(da aggiungere)
Flusso dei dati:	$Istruzione[31-0] \rightarrow \text{RIORDINO e SL1} \rightarrow (\text{Somma con PC}) \rightarrow \text{MUX} \rightarrow PC$	
Segnali di controllo:	Jump asserito per selezionare la nuova destinazione sul MUX	
	RegWrite=0 e MemWrite=0 per evitare modifiche a registri e MEM	
Tempo necessario:	Fetch e in parallelo il tempo dell'adder che calcola PC+4	
	(quindi il massimo tra i due tempi)	

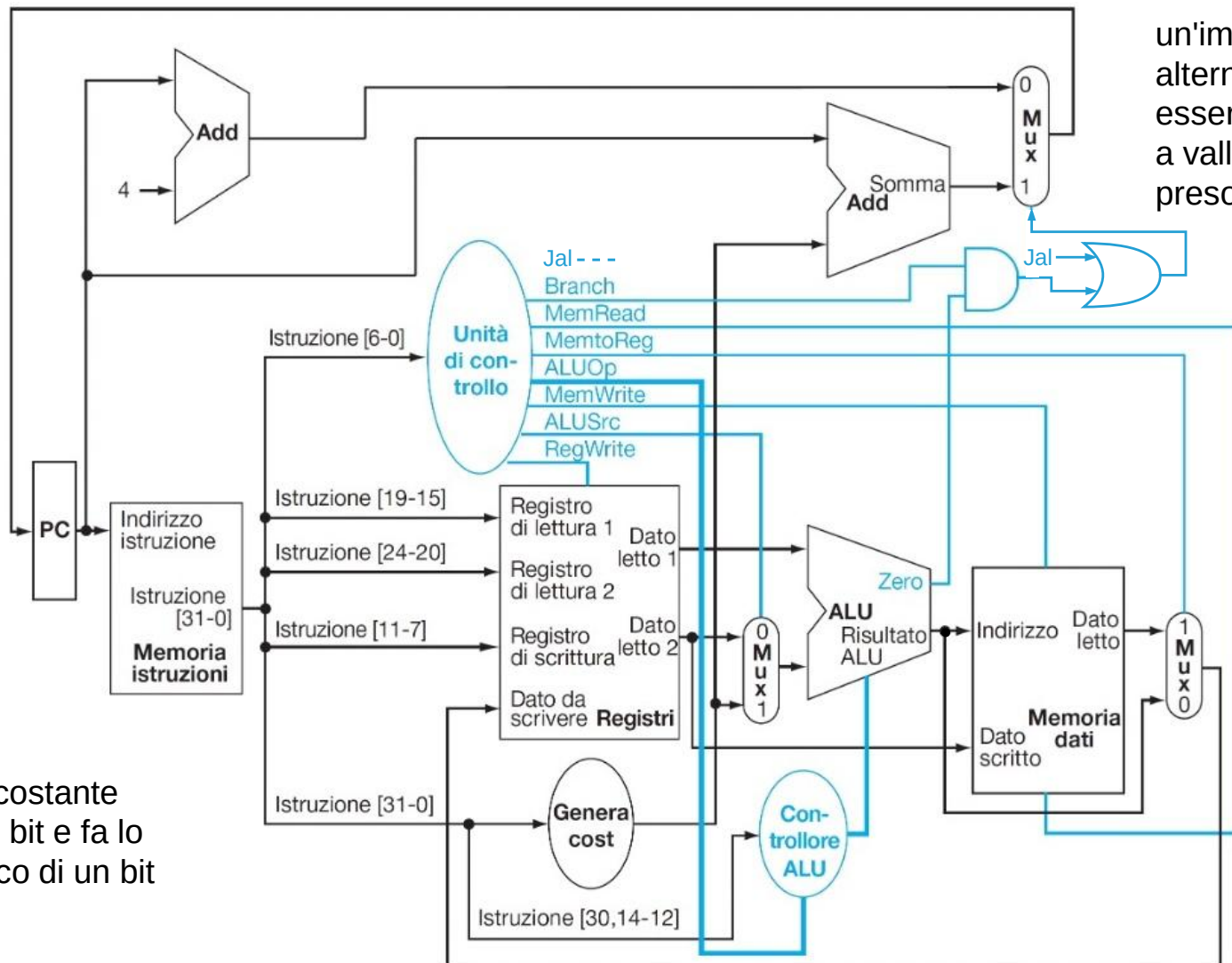
NOTA: come per il branch, l'hardware necessario al calcolo della destinazione del salto è sempre presente e calcola la destinazione anche se l'istruzione non è un Jump. Solo se la CU riconosce che è un Jump il valore calcolato viene immesso nel PC per passare (al colpo di clock successivo) alla destinazione del salto.

Obiettivo: una CPU più veloce possibile a costo di usare più componenti hardware

Aggiungere il jump (disegno)

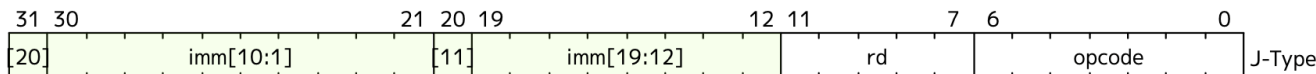


Aggiungere il jump



un'implementazione alternativa potrebbe essere con un MUX a valle e segnale preso dall'adder

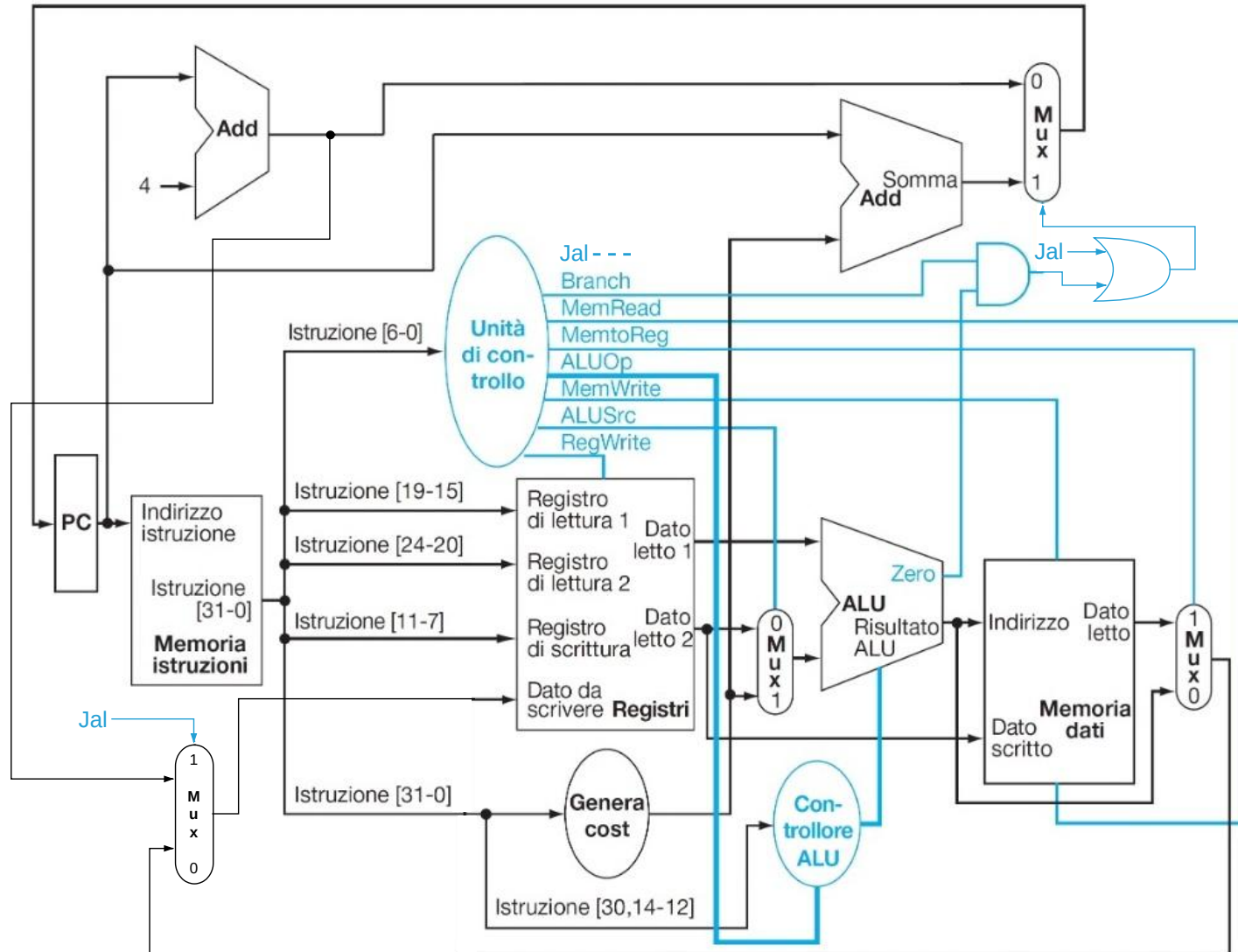
Genera costante riordina i bit e fa lo shift logico di un bit



Aggiungere il link

Cosa fa:	Quello che fa il Jump, e in più $rd \leftarrow PC + 4$	
Unità funzionali:	$PC + 4$	(già presente)
	MUX per selezionare il dato da scrivere	(da aggiungere)
Flusso dei dati:	Istruzione[31-0] $\rightarrow PC + 4 \rightarrow$ MUX $\rightarrow$ Dato da scrivere (banco registri)	
Segnali di controllo:	Jal asserito per selezionare la nuova destinazione sul MUX	
	RegWrite=1, MemWrite=0 per evitare modifiche a MEM	
Tempo necessario:	tempo del Jump in parallelo con il tempo dell'adder ($PC + 4$) + tempo per scrivere nei registri (quindi il massimo tra i due tempi)	

Jump and Link (Jal)



Istruzione addi/la

Assembly: **addi rd, rs1, costante** (add immediate) di tipo I

Cosa fa: Somma la parte immediata al registro **rs1** e pone il risultato in **rd**

Unità funzionali: **ALU** per la somma (presente)

MUX che seleziona la parte immediata come secondo arg (presente)

Estensione del segno della parte immediata (presente)

Flusso dei dati: Registri[rs1] → ALU

 Costante → Estensione segno → ALU

 ALU → Registri[rd]

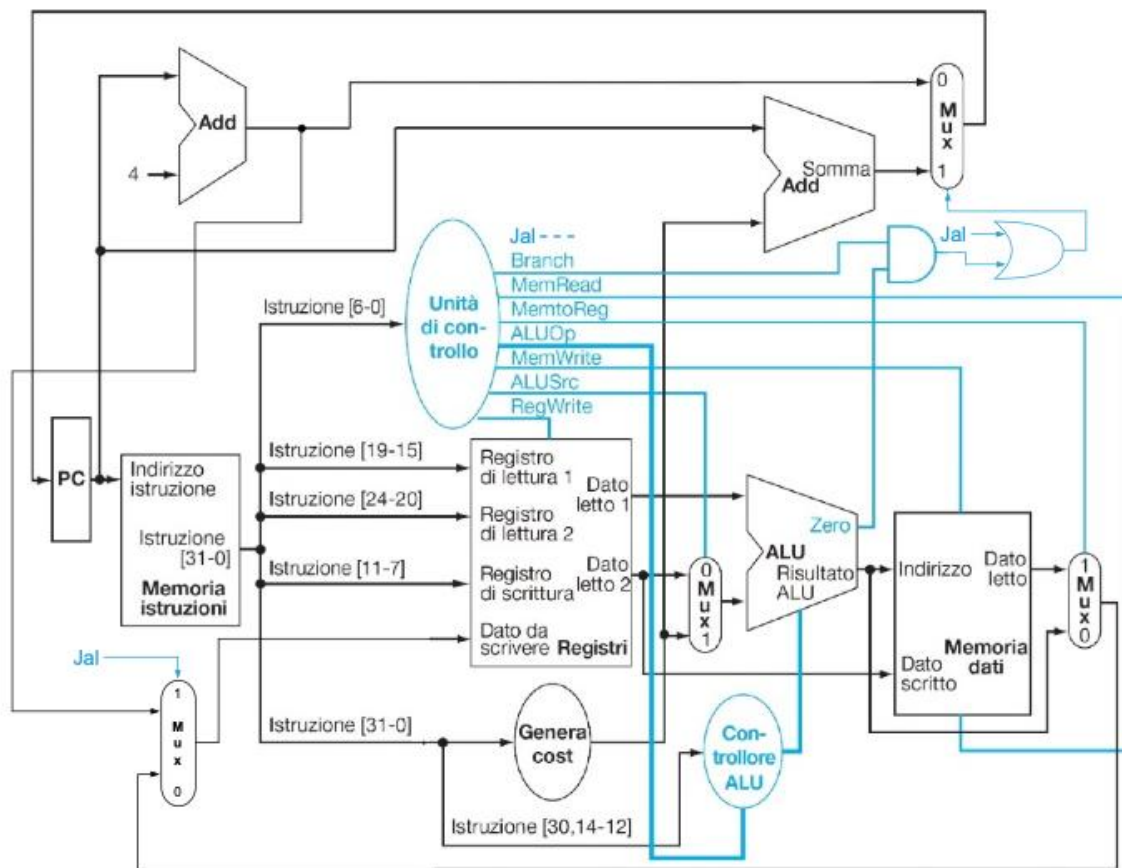
Si comporta quasi come una **lw rd, costante(rs1)** (ma lw memorizza l'indirizzo invece che il dato)

ed è uguale all'istruzione **la rd, label** (load address). La CU produce i segnali:

Istruzione	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	Jal	ALU Op1	ALU Op0
addi rd, rs1, costante	1	0	1	X	0	0	0	0	0

Tempo necessario: **come una istruzione di tipo R**

Istruzione addi/la: niente circuiteria aggiuntiva



Istruzione	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	Jal	ALU Op1	ALU Op0
<code>addi rd, rs1, costante</code>	1	0	1	X	0	0	0	0	0

Tempo necessario: come una istruzione di tipo R

Esercizio per casa



Aggiungere alla CPU l'istruzione **Jalr** (Jump Relative to Register)

di tipo I, che salta all'indirizzo (relativo al PC) contenuto nel registro **rs1 + immediate**, e inoltre che salva il valore di **PC+4** nel registro **rd**

- a) Modificare lo schema per realizzare l'istruzione (riciclando il più possibile)
- b) Indicare tutti i segnali di controllo che la CU deve generare
- c) Calcolare il tempo di esecuzione della istruzione assumendo che:

Accesso a memorie = 66ns, accesso ai registri = 33ns, ALU e sommatori = 100ns